

Project 01

Intelligent Systems II

Mário Antunes

2026

Projects

Form groups of one or two students and select **one** of the following projects. All projects will be hosted on **GitHub**, using GitHub Classroom.

The repository must contain all relevant scripts, configuration files, and a comprehensive README .md. Your README .md serves as your project report and must include instructions on how to run your agent, details regarding your solution's architecture, and an evaluation of its performance.

This is a four-week project (deadline 13/05/2026). You have until the end of this week to notify your professor (via e-mail) of your group members and chosen topic.

Do not forget to contact your professor with any questions. Further instructions may be added.

Grading

The project will be evaluated based on the following criteria:

Criteria	Description	Weight
Solution	Effectiveness and performance of the developed autonomous agent.	0.30
Code	Readability, maintainability, and adherence to best practices.	0.20
Repository	Organization of files, directories, and configuration.	0.20
Complexity	Complexity of the chosen game and the creativity of the proposed solution.	0.15

Criteria	Description	Weight
Report	Clarity of execution instructions, solution description, and evaluation.	0.10
Contributions	Identifying errors and providing solutions/fixes to the base simulation code.	0.05

Topics

The following projects are ordered by difficulty, from lowest to highest. You must select **only one**.

1. SI2 - Maze

- **GitHub Classroom Link:** [Join Maze Project](#)
- **Description:** SI2 - Maze is a simulation platform designed for developing and testing autonomous agents in grid-based environments. The platform provides a real-time visualization of agent behavior, allowing researchers and students to observe how different algorithms navigate complex mazes or explore entire rooms. The system follows a client-server architecture, where a Python-based backend handles the simulation logic and WebSocket communication, while an HTML5 Canvas-based frontend provides the visual interface.
- **Objective:** Agents are tasked with either reaching a specific target in “Maze Mode” or visiting every reachable tile in “Room Mode”. The simulation supports various configurations, including teleportation mechanics that allow agents to wrap around the edges of the grid. This flexibility enables the creation of diverse challenges, from simple pathfinding to exhaustive exploration in topologically complex environments.

2. SI2 - Wumpus

- **GitHub Classroom Link:** [Join Wumpus Project](#)
- **Description:** SI2-Wumpus is a modular simulation environment for the classic Wumpus World game, designed for testing and developing autonomous agents. The project features a Python-based WebSocket backend that handles the simulation engine, an HTML5 Canvas-based frontend for real-time visualization, and an extensible agent system that allows for easy implementation of custom AI strategies.

- **Objective:** The primary objective depends on the map type: in wumpus mode, the agent must find and collect the gold; in maze mode, it must reach a specific target; and in room mode, it must explore all reachable floor tiles. Agents can move in four cardinal directions (North, South, East, West) and can shoot arrows to kill the Wumpus, while avoiding deadly pits and the Wumpus itself.

3. SI2 - Battleship

- **GitHub Classroom Link:** [Join Battleship Project](#)
- **Description:** SI2 - Battleship is a modular simulation platform designed for developing and testing autonomous agents in the classic game of Battleship. The system features a centralized Python-based WebSocket backend that manages game logic, a real-time web viewer for monitoring matches, and an extensible framework for implementing custom AI strategies.
- **Objective:** The primary objective of the game is to sink all of the opponent's ships before they sink yours. Each player controls a 10x10 grid where their fleet is hidden from the opponent. Players take turns "firing" shots at specific coordinates, receiving immediate feedback on whether they hit a ship or splashed into the water, with successful hits granting an extra turn.

4. SI2 - Connect Four

- **GitHub Classroom Link:** [Join Connect Four Project](#)
- **Description:** Connect Four is a classic two-player connection board game in which the players choose a color and then take turns dropping colored discs into a seven-column, six-row vertically suspended grid. The pieces fall straight down, occupying the lowest available space within the column.
- **Objective:** The objective of the game is to be the first to form a horizontal, vertical, or diagonal line of four of one's own discs. In this project, we focus on developing autonomous agents that can play Connect Four against each other or against a human player. The game state is managed by a central server that communicates with the agents and a frontend viewer via WebSockets. Each agent receives the current board state and must decide the best column to drop their disc.

5. SI2 - Othello

- **GitHub Classroom Link:** [Join Othello Project](#)

- **Description:** Othello is a classic strategy board game played on an 8x8 grid. The game involves two players, Black and White, who take turns placing their discs on the board. This project provides a simulation environment featuring a backend server that manages the game state, a frontend for visualization, and a framework for developing autonomous agents.
- **Objective:** The primary objective is to out-position your opponent by “sandwiching” their pieces between your own, which allows you to flip them to your color. The player with the most discs of their color on the board when no more moves can be made wins the game. The backend handles the game logic, enforces rules, and communicates with connected agents via WebSockets, while the frontend provides a real-time view of the board and match statistics.

6. SI2 - Ultimate-TicTacToe

- **GitHub Classroom Link:** [Join Ultimate-TicTacToe Project](#)
- **Description:** Ultimate Tic-Tac-Toe is a strategic variation of the classic Tic-Tac-Toe game. It is played on a 9x9 grid, which is composed of nine 3x3 smaller grids, called “micro-boards”.
- **Objective:** The objective is to win three micro-boards in a row, column, or diagonal on the larger 3x3 “macro-board”. Players take turns placing their mark in one of the 81 empty cells. However, the position of a move in a micro-board determines which micro-board the next player must play in. For example, if a player moves in the top-right cell of a micro-board, the next player must play in the top-right micro-board. If a micro-board is already won or full, the next player is granted a “free move” and can play in any available cell on the entire board.

GitHub Classroom Access

Here are detailed instructions to access GitHub Classroom and set up your repository.

1. Join the Assignment and Form Your Team

1. **Access the link:** Click the specific GitHub Classroom link for your chosen topic provided in the section above.

2. **Find your name:** Select your name from the student list. > **Can't find your name?** All names registered on PACO were added. If yours is missing, please contact [Prof. Mário Antunes](#).
3. **Create a team (ONE member only):** Only **one** person from your group should create a team. Follow this exact naming structure (the nmec should be sorted): [nmec1]_[nmec2]_[topic_name]
 - (Example: 132745_133052_maze or 132745_wumpus)
4. **Join the team (Second member, if applicable):** The second project member must find and join the team created in the previous step.

2. Access the Organization and Repository

1. **Accept the email invite:** After joining a team, all members will receive an email invitation to join the detiuaveiro GitHub organization.
2. **You must accept this invitation** before you can continue.
3. **Refresh the page:** Go back to the GitHub Classroom page and refresh it.
4. **Verify access:** You should now see and have access to your team's working repository.

3. Configure an SSH Key for Access

This will allow you to clone and push to the repository from your command line without entering your password every time.

1. **Check for an existing SSH key:** Open your terminal and run this command:

```
cat ~/.ssh/id_ed25519.pub
```

2. **Generate a key (if needed):**

- If you see a key (starting with ssh-ed25519...), copy the entire line and skip to step 3.
- If you see an error like “No such file or directory,” run the following command to create a new key:

```
ssh-keygen -q -t ed25519 -N ''
```

- After it's generated, run `cat ~/.ssh/id_ed25519.pub` again to view your new key and copy it.

3. **Add the key to your GitHub account:**

- Go to your GitHub **Settings**.
- On the left menu, click **SSH and GPG keys**.

- Click the **New SSH key** button.
- Give it a **Title** (e.g., “My UA Laptop”).
- Paste the key you copied into the **Key** field.
- Make sure the “Key type” is set to **Authentication Key**.
- Click **Add SSH key**.

4. Authorize the key for SSO:

- After adding the key, find it in your list on the same page.
- Click **Configure SSO**.
- Select the **detiuaveiro** organization, fill in your login details, and grant access.